SOFTWARE REQUIREMENTS SPECIFICATION

SNAKE GAME

A simple 2D arcade game project

Project Name	Snake Game
Document Type	Software Requirements Specification (SRS)
Version	1.0
Prepared For	Academic / Mini Project
Prepared By	Student Name: _____
Department / Class	_____
Date	_____

Document Overview

This document defines the requirements for a basic Snake Game. It can be used as a guide for designing, developing, testing, and demonstrating the project.

Table of Contents

• 1. Introduction

• 2. Overall Description

• 3. System Features

• 4. External Interface Requirements

• 5. Non-Functional Requirements

• 6. Hardware and Software Requirements

• 7. Use Case and Game Flow

• 8. Testing Requirements

• 9. Limitations and Future Enhancements

• 10. Acceptance Criteria

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to describe what the Snake Game should do and the conditions under which it should operate. It gives developers and testers a clear reference for building and checking the game.

1.2 Scope

The Snake Game is a single-player 2D game. The player controls a snake using keyboard keys, collects food, and earns points. The snake grows after eating food. The game ends if the snake hits a wall or its own body.

1.3 Intended Audience

This document is intended for students, project guides, developers, testers, and anyone reviewing the mini project.

1.4 Definitions and Abbreviations

• **SRS:** Software Requirements Specification.

• **GUI:** Graphical User Interface.

• **FR:** Functional Requirement.

• **NFR:** Non-Functional Requirement.

2. Overall Description

2.1 Product Perspective

The game is a standalone desktop application intended for learning and entertainment. It does not require a network connection for basic gameplay.

2.2 Main Product Features

• Start and play a new game.

• Move the snake up, down, left, and right.

• Generate food at random positions.

• Increase the snake's length and score when food is eaten.

• Detect collisions and show a Game Over message.

• Allow the player to restart or exit the game.

2.3 User Characteristics

The intended user is anyone who can use basic keyboard controls. No special training is required.

2.4 Assumptions and Dependencies

• The computer has a working display and keyboard.

• The operating system supports the selected programming language and graphics library.

• The player understands the basic game instructions.

3. System Features

3.1 Game Start

FR-01: The system shall display a start screen or begin a new game when requested.
FR-02: The system shall initialize the snake, score, and first food position.

3.2 Snake Movement

FR-03: The system shall move the snake continuously in the current direction.
FR-04: The system shall accept arrow-key input (or configured equivalent) to change direction.
FR-05: The system shall prevent an immediate reversal into the snake's opposite direction, where applicable.

3.3 Food and Scoring

FR-06: The system shall place food at a valid random location on the game board.
FR-07: When the snake eats food, the system shall increase the snake's length.
FR-08: The system shall increase and display the player's score after food is eaten.

3.4 Collision and Game Over

FR-09: The system shall detect collision with the boundary of the game board.
FR-10: The system shall detect collision between the snake's head and its body.
FR-11: When a collision occurs, the system shall stop gameplay and display the final score.

3.5 Restart and Exit

FR-12: The system shall allow the player to restart the game after Game Over.
FR-13: The system shall provide a way to close the game safely.

4. External Interface Requirements

4.1 User Interface

• The game board shall be clearly visible.

• The snake and food shall use distinguishable visual styles.

• The current score shall be shown during gameplay.

• Game Over and restart instructions shall be easy to understand.

4.2 Hardware Interface

The game uses a standard computer display and keyboard. No special hardware is required.

4.3 Software Interface

The game may be implemented using Python with Pygame, Java, C++, or another suitable language and graphics library. The final implementation must match the chosen environment.

5. Non-Functional Requirements

• **Usability:** Controls and instructions should be simple for beginners.

• **Performance:** Movement and score updates should appear smooth on a typical student computer.

• **Reliability:** The game should handle normal gameplay without unexpected crashes.

• **Maintainability:** Code should be organized so that features can be corrected or improved.

• **Portability:** The game should run on the operating system supported by the chosen development tools.

• **Security:** The basic game should not need sensitive personal information or online accounts.

6. Hardware and Software Requirements

6.1 Hardware Requirements

• Desktop or laptop computer.

• Keyboard and display.

• Basic memory and processor resources sufficient for the chosen development environment.

6.2 Software Requirements

• Operating system: Windows, Linux, or macOS, depending on implementation.

• Programming language and runtime: selected by the developer.

• If using Python: Python 3 and the Pygame library.

• Code editor or IDE, such as VS Code or another suitable editor.

7. Use Case and Game Flow

7.1 Main Use Case

Actor: Player
Precondition: The game is installed or its source code can be run.
Main flow: Open game → Start → Control snake → Eat food and gain points → Avoid collisions → View final score.
Alternative flow: After Game Over, restart or exit.

7.2 Simple Game Flow

Step	Description
1	Launch the game
2	Initialize snake, score, and food
3	Read keyboard input and move snake
4	Check whether food is eaten
5	If food is eaten, grow snake and update score

Step	Description
6	Check wall and self-collisions
7	If collision occurs, show Game Over; otherwise repeat movement

8. Testing Requirements

The following test cases should be checked before the project is accepted.

Test ID	Test	Expected Result
TC-01	Launch the game	Game opens and initializes correctly
TC-02	Press direction keys	Snake moves in the selected valid direction
TC-03	Eat food	Snake grows and score increases
TC-04	Hit a wall	Game Over appears and movement stops
TC-05	Hit own body	Game Over appears and final score is shown
TC-06	Restart after Game Over	A new game starts with reset score and snake
TC-07	Exit the game	Application closes safely

9. Limitations and Future Enhancements

9.1 Current Limitations

• The basic version supports one player.

• The game may not save scores after the application closes.

• Graphics and sound may be minimal.

9.2 Future Enhancements

• Add difficulty levels and adjustable speed.

• Save high scores.

• Add sound effects and improved graphics.

• Add pause/resume functionality.

• Add a mobile or touch-screen version.

10. Acceptance Criteria

The project is considered complete when the player can start the game, control the snake, collect food, see the score increase, trigger Game Over through a collision, and restart or exit without errors.

References

This SRS is an original, student-friendly specification prepared for a basic Snake Game mini project. The final requirements should be updated to match the actual programming language, controls, and features implemented.